



Scoring the Migration

Zach Kelling

Lux Industries

2026

Scoring the Migration

Does the proposed Lux end-state distribute the powers, or rename the multisig. The rubric is applied to our own proposal, and to our own stack, on the same terms as Morpheus.

1 What is being scored

The target is Morpheus as the adoption series proposes to leave it: upgrade authority behind `lux/dao` (a Safe with a Governor module and a timelock delay), a vote-escrow token `vMOR`, settlement moved to `mord` on a Lux L2, with the L2 anchoring to Lux primary-network finality. The question is not whether that is better than the anonymous 5-of-9 Morpheus runs today. It is whether the end-state actually distributes the six powers, or whether it moves them from a multisig to a stack that one party still controls under better names. This is graded from the code the proposal names, not from the proposal's prose, and the same standard is turned on the Lux contracts and nodes that would host it.

Scores are read best-faith: where a power depends on a migration step (transfer `owner()` to the timelock, remove the Safe's direct signers, open the validator set), the score assumes the step is completed and is marked conditional[†]. The heal-thyself section then reports which of those steps our own live deployments have not in fact taken, because a conditional score the sponsor never satisfies is theater by the rubric's own definition.

2 The vector

Power	Score	Measured state (file:line)	What the score depends on
Upgrade	1 [†]	<code>owner()</code> moves to a delay; but the "timelock" is a module parameter the owner tunes (<code>ModuleGovernorV1.sol:404 updateTimelockPeriod onlyOwner</code>) and the module is UUPS-upgradeable by its owner (<code>:200</code>)	Safe signers removed and module owned by the DAO, not by a multisig that can still sign directly
Custody	1 [†]	<code>MOR</code> mint stays uncapped, now behind the delay (<code>MOROFT.sol:50,61</code> , unchanged by the migration); session escrow returns on close; slashing is app-level in <code>mord</code> , not primary-level	A supply cap, and the delay actually gating the owner-minter path
Halt / censor	0	An L2 from <code>l2-template</code> produces blocks from one operator's five pods (<code>init.sh node.yaml replicas:5</code> , one namespace; MPC and bridge single-replica); ADOPT-01 states "Validators are Lux-managed"	Opening the L2 producer set to independent operators; a forced-inclusion / escape path
Governance	1 [†]	Governor to timelock to execution exists, but <code>vMOR</code> weight is flat 1:1 (<code>VotesERC20StakedV1.sol stake to _mint</code>), quorum is 4% [<code>zoo.json</code>], and 76% of emission is owner-directed (<code>DistributionV6.sol:175 onlyOwner</code>)	A vote the emission-holder cannot alone carry; Safe removed so execution is vote-gated
Exit	1	Code is open and forkable; <code>MOR</code> is self-custodied on Ethereum/Arbitrum/Base; funds bridged onto the single-operator L2 have no withdrawal independent of that operator	An escape hatch from the L2 that does not require the operator to be live
Liveness	0	The compute L2 halts if the one operator's cluster, MPC signer, or bridge stops; the base-chain <code>MOR</code> contracts are unaffected	Many independent validators/RPCs for the chain that carries settlement

[[†]conditional: assumes a migration step our live reference deployments have not performed; see heal-thyself.]

Vector $[1^\dagger, 1^\dagger, 0, 1^\dagger, 1, 0]$. The real-decentralization ceiling is the lowest score among load-bearing powers. Halt and Liveness are 0, so the ceiling of the migrated compute chain is **0**, the same ceiling as Morpheus today. The migration’s genuine gains sit on the authority axes (Upgrade, Custody, Governance move from 0 toward 1), and every one of those gains is conditional on a step the sponsor has not yet taken in its own production stack.

3 The two zeros set the ceiling

The proposal’s strongest security claim is inherited post-quantum finality from Quasar. That claim is about finality, which is the order and irreversibility of blocks once produced. It is not about who produces them. An L2 stood up by `l2-template` is a Kubernetes `StatefulSet` of five node replicas in one operator’s namespace, beside a single MPC signer and a single bridge [verified: `l2-template/init.sh`]. ADOPT-01 is explicit that “Validators are Lux-managed” [verified: `adopt-01`]. Both readings fail the client’s own test: if Lux runs the validators it is one org, and if Morpheus runs the five pods it is one org. The permissionless staking of the Lux *primary* network (anyone meeting `MinValidatorStake` may validate via `AddPermissionlessValidatorTx`, [verified: `node/vms/platformvm`]) does not transfer to the L2’s producer set; the L2 recognizes primary-network finality but sequences its own blocks. So Halt/censor is 0: one operator produces, a user can be excluded, and there is no forced inclusion. Liveness is 0 for the same reason: the chain that carries settlement stops when that one operator stops.

This is not worse than Morpheus today only because Morpheus today runs on Base, which is itself a single-sequencer rollup. The migration does not fix the axis; it moves the single sequencer from Coinbase’s infrastructure to one org’s five pods. Two independent facts compound it: Lux primary staking has no slashing of principal (`grep -ri slash node/vms/platformvm node/snow` is empty [verified]), so even the finality layer’s validators are bonded but not slashable; and the realized stake distribution, the number that would set the primary network’s Nakamoto coefficient in practice, is not something the repository establishes [unmeasured: no on-chain census in-tree]. We do not assert a coefficient we have not counted.

4 Governance is binding but capturable

Take the migration as completed: `owner()` sits behind the timelock, the Safe no longer signs directly, and every mint, upgrade, and pool edit is propose, vote, delay, execute. That is a real improvement over an off-chain forum, and it earns the 1. It does not earn the 2, and the reason is the specific mechanism, not a distribution that “might” thin out.

`vMOR` is named for vote-escrow and the token-design note describes weight “scaled by lock length,” so that “a long holder and a mercenary one weigh differently” [proposal: `mor-token-design.md`]. The contract the proposal names, `VotesERC20StakedV1`, does not do this. `stake(amount)` mints voting power one-to-one with the amount staked (`_mint(msg.sender, amount_)`), and the only time constraint is a single global `minimumStakingPeriod` checked at `unstake` [verified: `repo`]. There is no per-lock duration and no duration-weighting anywhere in the contract set [verified: no lock-length term in `dao/contracts`]. Vote weight is therefore proportional to MOR staked, full stop.

That matters because of where MOR comes from. Seventy-six percent of emission is assigned by the owner through `manageUsersInPrivatePool`, an `onlyOwner` call (`DistributionV6.sol:175` [verified]). Quorum in the live config is 4% [verified: `zoo.json`]. A vote whose weight is proportional to a balance the emission controller allocates, at a 4% quorum, is a vote the emission controller can meet alone. The capture vector the design says it closes is left open by the contract the design cites. Under the rubric’s own Governance test, “insiders alone meet quorum” is the theater case; the timelock and the on-chain execution are what keep this at 1 rather than 0, but the 1-to-2 step, a vote no insider can carry, is not delivered and is undermined by the flat weighting.

5 Upgrade and custody: real progress, still conditional

The authority axes are where the migration does honest work, and it is worth stating plainly. Today Morpheus has no execution delay anywhere in either contract tree and no vote surface at all; the compute diamond is an `OwnableDiamond` (`LuminDiamond.sol:9`) whose owner can recut

any facet in one transaction, and MOROFT mint is uncapped behind an owner-set minter (:50,61) [verified]. Putting `owner()` behind a timelocked Governor converts each of those one-key actions into a delayed, visible, on-chain process, which is the 0-to-1 move and a genuine one.

The conditions are the whole story. Upgrade reaches 1 only if the Safe’s direct signers are removed, because the `lux/dao` timelock is not a standalone `TimelockController`: it is the Azorius module’s internal `timelockPeriod`, and both that period (`updateTimelockPeriod, onlyOwner`) and the module itself (`UUPS _authorizeUpgrade, onlyOwner`) answer to the module owner (`ModuleGovernorV1.sol:404,200` [verified]). If the owner is a Safe whose signers can still sign, the delay is a setting that same group can zero. Custody reaches 1, not 2, because the migration explicitly does not change MOROFT; the uncapped owner-minter path survives, now behind the delay rather than removed or capped. `mord` adds slashing, but at the application layer for providers, not to the validators that secure the chain.

6 Heal-thyself: the conditions our own stack has not met

The conditional scores above assume steps our production deployments have not taken. Disclosing this is the point: selling “real decentralization” while our own chains retain admin authority would be the exact hypocrisy the client is trying to avoid.

Our production timelock keeps its deployer as admin.

The Lux governance deploy constructs `Timelock(1 days, [deployer], [address(0)], deployer)` (`standard/contracts/script/DeployMultiNetwork.s.sol:254` [verified]). The deployer is the sole proposer and the retained admin role, and the script never renounces it (`grep` for `renounce/revokeRole` is empty). `vLUX` and the `GaugeController` are deployed but not wired as proposers. As scripted, that is a one-day delay a single EOA administers and alone can propose into: the governance contracts exist, the vote does not gate execution. This is the same finding we level at Morpheus, in our own tree. [verified: deploy script; on-chain role state not independently re-confirmed here]

Our launch multisig is one seed.

`governance.md` specifies all five launch signers derived from a single `LUX_MNEMONIC`, with rotation to independent hardware keys planned before a deadline (2026-08-08) that has now passed, status “TBD” [verified]. Until rotation, the effective control set of the launch Safe is one, which is the Ronin shape the rubric names. The doc is candid that this is time-bounded bringup, which is honesty, not theater; but the rotation is a claim about the future, not a measured state.

Our verified settlement is not yet verified.

We would offer `mord` and `lux/ai` as the un-fakeable base. The current `lux/ai` settlement path, `aivm.go SubmitResult`, builds a reward receipt from a self-reported result (provider id, compute time, output) and calls `SubmitReceipt (:311)` with no signature, countersignature, attestation, or quorum check. The attestation verifier runs only at provider registration (:216,223), and `SettleQuorum` (`rewards.go:407`) is off this path; the miner polls the public `/v1/bc/A/ai/pendingTasks` and is paid the self-reported reward (`miner.go`) [verified]. The L2 strategy note already flags this and orders it first [disclosed: `morpheus-as-lux-l2.md`]. It is disclosed, and it is not yet fixed.

Our second governance tenant is a front end.

ADOPT-03 cites `zoo/vote` as the stack “white-labeled through one config,” implying a live instance. Its `config/zoo.json` has every contract address null: `safeFactory`, `moduleGovernor`, `governor` [verified]. It is a branded UI pointing at no deployed contracts, not a running governance deployment.

7 Honesty flag and verdict

Honesty flag: mixed. The architecture-level trade-offs are disclosed. The notes state that the L2 surrenders producer-set control to a Lux-managed set, that the settlement bypass must be fixed first, and that a raise clears only at real demand [disclosed]. Those are immaturity, not theater. Three claims are not disclosed and are contradicted by the code: `vMOR` presented as duration-weighted when the named contract is flat 1:1; “MOR becomes credibly decentralized” and “real governance” presented as delivered when our own reference stacks retain a deployer-admin timelock

and an operational Safe; and `zoo/vote` presented as a live tenant when its addresses are null. A property claimed but not present in the code is the gap the rubric exists to measure.

Verdict: theater on the load-bearing powers, genuine but conditional progress on the rest. The mechanism exists. It does not, as specified, distribute the powers that carry funds and liveness. Halt and Liveness stay at 0 because settlement moves onto a single-operator L2. Governance is binding but capturable by the party that already directs 76% of emission, because the vote is weighted by a balance that party allocates. Upgrade and Custody improve from 0 toward 1, conditional on removing an admin our live deployments still hold. The net effect of the migration as drawn is to replace an anonymous 5-of-9 with a named stack that a single operator and a single emission-holder can still control: centralized under a nicer name, which is precisely the outcome the client asked us to avoid.

Three changes make it real, and none is delivered by the default path. *Open the L2 producer set* to independently operated validators, or stop describing finality-anchoring as validator decentralization. *Make the vote capture-resistant*: weight it by committed lock duration as the design claims, and set quorum above the owner-directed emission share, or the emission controller is the government. *Actually renounce the admin*: remove the Safe’s direct signers and renounce the timelock’s deployer-admin role, on Lux first, so the delay we sell is one we ourselves cannot bypass. Until those land, the migration is a better starting point than Morpheus has today, honestly graded as a 1-ceiling proposal with two load-bearing zeros, not the finished decentralization it reads as.

RUBRIC APPLIED. Method in “Real Decentralization”; governance path in ADOPT-03; settlement in ADOPT-02; the L2 trade in ADOPT-01. • research@lux.network